

专题：map / odom / base_footprint——坐标系与初始位姿全解析

做导航时最容易绕晕的就是那一堆坐标系和参数：Nav2 配置里有 `global_frame_id: map`，地图 YAML 里有 `origin: [-3.89, -2.95, 0]`，命名点表里有 `initial_pose: (0, 0, 0)`，RViz 里还能点「2D Pose Estimate」……它们到底是什么关系？本专题一次讲透。

与其他讲义的分工：本篇回答「坐标系是什么、原点在哪、初始位姿给谁看」这类**静态概念**问题；`map` → `odom` 这条边运行时怎么被不断结算（漂移与矫正的动态过程），见 `docs/10.3.3_专题_里程计漂移与矫正.md`；导航主线四步见 `docs/10.3.3_Nav2导航系统与路径规划.md`。

本文改编并扩展自 [ROS 2 Navigation2 坐标系统全解析（CSDN, roseey）](#)，所有数值、参数、文件路径均已替换为本项目实物。

0. 一图先行：三个坐标系与四个易混概念

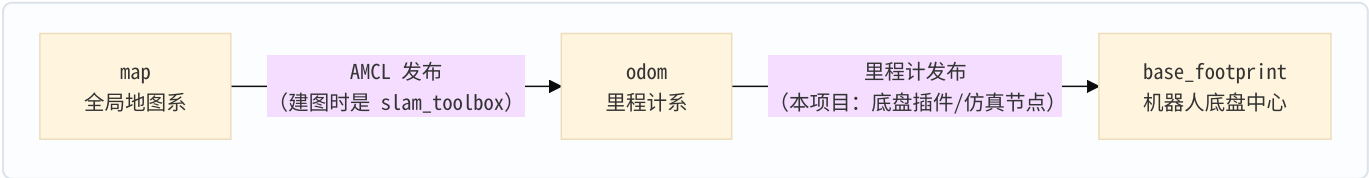
map ——(AMCL 发布，低频可跳)——▶ odom ——(里程计发布，高频连续)——▶ base_footprint

概念	它是什么	谁决定的	什么时候定的
map 原点 (0,0)	全局坐标系的零点	建图启动瞬间机器人的落脚点	建图时，一次定死
地图 YAML 的 <code>origin</code>	栅格图片左下角像素在 map 系的坐标	建图轨迹自动算出	存图时，写进文件
<code>initial_pose</code>	告诉 AMCL 「机器人现在大概在哪」	人 / 启动脚本	每次导航启动时
2D Pose Estimate	运行时重发一次 <code>initial_pose</code>	你在 RViz 里点的那下	随时

前两个属于「地图本身」，建完图就冻结；后两个属于「定位初值」，每次启动都要给。把这两组概念混为一谈，是本专题要拆掉的第一个误区。

1. 三个关键坐标系：为什么是三个，不是一个

Navigation2 最核心的是这条 TF 链：



坐标系	由谁发布 TF	含义	原点在哪
map	AMCL (建图阶段是 slam_toolbox)	全局地图坐标系， 永不漂移	建图启动时机器人的落脚点
odom	里程计 (Gazebo 的 MecanumDrive 插件 / 轻量后端的 simulator_node)	局部里程坐标系， 随时间漂移	机器人上电 (仿真启动) 时的落脚点
base_footprint	固定在机器人上 (由 URDF/robot_state_publisher 维护向下的链)	机器人底盘在地面的投影中心	机器人本体

命名小注：REP-105 里标准帧叫 base_link (刚体中心，可能离地有高度)；base_footprint 是它在地面的投影，2D 导航常用后者。本项目全栈统一用 base_footprint (见 mecamind_nav2_params.yaml 里所有 robot_base_frame)。

为什么需要三个？这是 ROS 导航社区多年沉淀的最佳实践，核心是**两类信息源的性格互补**：

- odom → base_footprint：轮速积分得到。**短期精确、高频连续，但长期漂移**——误差只进不出，越走越歪。
- map → base_footprint (合成结果)：激光对照地图矫正。**长期精确、不累积误差，但修正时位姿会「跳一下」**。
- 控制器 (DWB, 10 Hz) 只消费连续的 odom → base；规划器和任务层消费准确的完整链 map → base。两类消费者各取所需，互不干扰。

AMCL 做的事情就是不断计算 map → odom 这条边，把「会漂的里程计」钉到「不会动的地图」上。这条边等于「激光估计的真实位姿」与「里程计位姿」的差，公式与实测阶梯曲线见《里程计漂移与矫正》专题第 10 节。

2. map 原点在哪里？——建图那一刻就永久定了

答案：SLAM 启动的瞬间，机器人当时的落脚点被定义为 map 系的 (0, 0, 0)，此后永不改变。

2.1 图解建图过程

```
第二课建图开始
↓
slam_toolbox 启动
↓
此时机器人所在位置 = map 坐标系 (0,0)  ← ☑ map 原点在这里确定！
↓
机器人在三居室里绕一圈，SLAM 记录所有墙和家具
↓
保存地图：mecamind_three_room_nav.pgm + .yaml
```

本项目建图从客厅中央（两个后端统一的出生点）启动，所以三居室地图的 (0,0) 就在客厅中央。**map 坐标系一旦建完图就彻底固定**——跟第三课导航时你在 RViz 里怎么点、机器人从哪启动，完全无关。

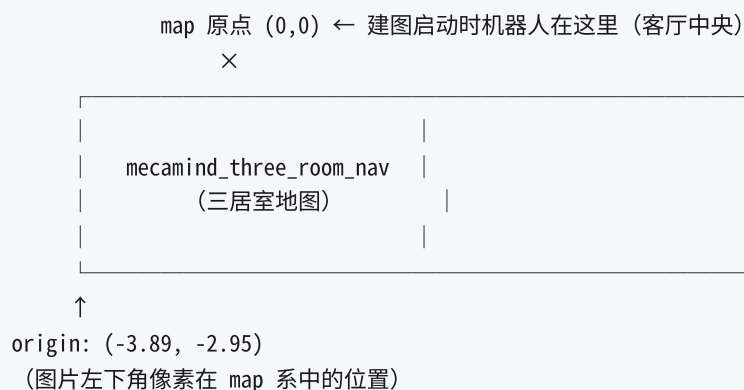
由此推论：`mecamind_named_goals.yaml` 里所有坐标都是这个「规定」的产物——bedroom 的坐标含义就是「建图出发点往某方向多少米」。换个位置建图，同一间屋子会得到一张所有坐标全部不同的地图，命名点、巡逻点全部作废。这就是工程上要约定固定建图起点的原因。

2.2 YAML 里的 origin 是什么？（最高频误区）

看本项目的真实地图元数据：

```
image: mecamind_three_room_nav.pgm
mode: trinary
resolution: 0.05000000074505806
origin:
- -3.8935666119002796
- -2.9538319511793265
```

`origin` 的意思是：地图图片左下角的那个像素（图片自身坐标的 0,0），对应到 `map` 坐标系中的坐标是 (-3.89, -2.95)。画一张图就懂了：



origin 不是 map 原点。map 原点永远是 (0,0)；`origin` 只是「地图纸张左下角」在 `map` 系里的坐标——它是建图工具存图时自动算出来的结果，不是谁手填的参数：建图时机器人从客厅中央出发，往左最远画到 3.89 m、往下最远画到 2.95 m 处的墙，栅格左下角自然落在那里。

它的唯一作用是让 Nav2 知道「图片上每个像素对应 `map` 系的哪里」。米 ↔ 像素的换算只有一行：

$$\begin{aligned} \text{列} &= \left\lfloor \frac{x - \text{origin}_x}{\text{resolution}} \right\rfloor \\ \text{行} &= \left\lfloor \frac{y - \text{origin}_y}{\text{resolution}} \right\rfloor \end{aligned}$$

这条换算贯穿导航全栈：AMCL 给粒子打分查格子、代价地图膨胀、NavFn 波前扩散，全在这套「米 ↔ 格」换算上跑（代入实数的手算例子见《里程计漂移与矫正》第 4.1 节）。

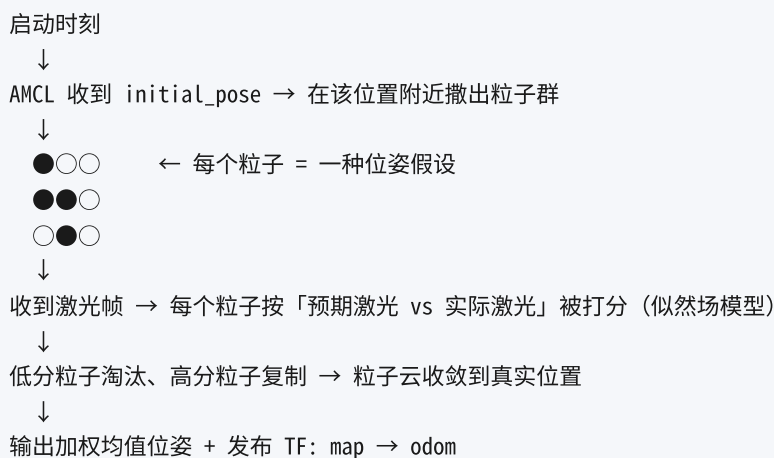
3. initial_pose 是什么？——给 AMCL 的「寻人启事」

map 原点讲清了，现在换一个问题：导航启动时，机器人可能被放在地图上任何地方，AMCL 凭什么知道它在哪？

答案是：它不知道，必须有人告诉它一个大概位置——这就是 `initial_pose`：「机器人现在大概在地图的这个位置附近，请从这里开始找。」

3.1 AMCL 的工作原理（粒子群）

AMCL 内部维护一群「粒子」（本项目 500~2000 个，见 `mecamind_nav2_params.yaml` 的 `min_particles` / `max_particles`），每个粒子是一个假设：「如果机器人在这里、朝这边，激光应该看到什么」。



打分、重采样、`OmnimotionModel` 全向运动模型（麦轮必须用它，差速模型会把横移当噪声）的细节，见《里程计漂移与矫正》第 10.3~10.4 节。

3.2 不给 initial_pose 会怎样？

分两种情况：

情况 1：机器人刚好在 map 原点附近（本项目仿真正是这样）

仿真把机器人放在出生点 (0,0)
建图也是从这个出生点启动的 → map 原点 = 出生点
`initial_pose (0,0,0)` 恰好等于真实位置
→ AMCL 几秒内收敛 → 导航正常 ☑

`mecamind_named_goals.yaml` 里把 `initial_pose` 写成 (0, 0, 0)，就是这个逻辑：

```
# 第二课的地图从原点开始建，map 坐标系与世界坐标系近似重合，
# 因此初始位姿与两个后端统一的出生点 (0, 0) 一致。
initial_pose:
  frame_id: map
  x: 0.0
  y: 0.0
  yaw: 0.0
```

这是把两课的约定串起来的结果，不是巧合——建图起点、仿真出生点、初始位姿三者被刻意对齐了。

情况 2：机器人不在 initial_pose 附近

```
粒子群在 (0,0) 附近散开
但机器人实际在 (5,3)
所有粒子的「预期激光」都与实际激光对不上
→ 所有粒子分数都很低 → AMCL 不知道该留谁淘汰谁
→ 位姿乱跳 / TF 不发布 / 导航完全失败 ×
```

这就是主讲义 §9.1「目标一发就 failed，日志报 Initial robot pose is not available」和 §9.2「机器人瞬移、粒子云发散」的病根。

3.3 AMCL 不能自己在整张地图里找机器人吗？

可以，但默认没开。看本项目参数：

```
# 恢复注入（定位丢失时撒随机粒子）关闭：教学场景地图小、初始位姿已知
recovery_alpha_fast: 0.0
recovery_alpha_slow: 0.0
```

两者都为 0.0 = **全局恢复被禁用**。恢复机制的含义是：当所有粒子分数都极低（「我在哪都对不上」）时，以一定概率往地图的自由空间里重新撒随机粒子——相当于在全图「碰运气搜索」。

为什么默认禁用？

优点	缺点
被「绑架」到地图任意位置后能自救	效率低，可能很久才碰巧找到
	对称环境（长得一样的房间/走廊）容易自信地认错
	大地图里可能永远找不到

工程上的标准做法：**启动时给一个 ±1 m 以内的初始估计**，而不是让 AMCL 在全图靠运气。粒子滤波本质是**局部收敛算法**——初值给对了，几步收敛到厘米级；初值错了房间，就在错误位置附近打转，永远追不回来（原理展开见《里程计漂移与矫正》第 7 节）。

3.4 协方差：告诉 AMCL 你有「多不确定」

`initial_pose` 消息（`PoseWithCovarianceStamped`）除了位姿还带一个协方差矩阵——你对这个估计的不确定度：

- `x/y 方差 0.25` = 位置有约 ± 0.5 m 的不确定度（方差 = 标准差²）；
- `yaw 方差 0.0685` $\approx$ 角度约 $\pm 15^\circ$ 的不确定度；
- 设太小 $\rightarrow$ 粒子挤成一个点，初值稍偏就纠不回来；
- 设太大 $\rightarrow$ 粒子撒得太散，收敛很慢。

协方差是 6×6 矩阵按行展开的 36 个数，x 方差在下标 0，y 方差在下标 7（第 2 行第 2 列），yaw 方差在下标 35——本项目 `lifecycle_activator.py` 的注释里有逐项解释。

4. RViz 的「2D Pose Estimate」在做什么？

不是在设置 map 原点。很多人在这里搞错。它做的事和 §3 的 `initial_pose` 完全相同，只是换了个入口：

```
你在 RViz 地图上按下并拖动（点 = 位置，拖 = 朝向）
↓
RViz 向 /initialpose 话题发布一条 PoseWithCovarianceStamped
↓
AMCL 收到：「用户说机器人现在在 map 系的这里」
↓
AMCL 把整群粒子搬到该位置附近重撒
↓
激光对照地图打分 → 粒子云收敛 → 继续发布 map → odom
```

这只是重置 AMCL 的估计起点，不改变 map 坐标系分毫。

生活类比：一张北京地图

ROS 概念	地图类比
map 坐标系	北京的经纬度系统（固定的）
map 原点 (0,0)	建图时你站的地方，比如天安门广场
YAML 的 <code>origin</code>	「地图纸张左下角」对应哪个经纬度
<code>initial_pose</code>	「我现在大概在王府井附近，帮我确认下具体位置」
2D Pose Estimate	你用手指指着地图说「我在这里」

你用手指指地图，并不会改变北京的经纬度系统。

5. 实际工程中初始位姿怎么给？三种方式

方式 1：YAML 里固定（适合固定出发点）

Nav2 的 AMCL 支持在参数里写死：

```
amcl:
  ros__parameters:
    set_initial_pose: true
    initial_pose.x: 0.0
    initial_pose.y: 0.0
    initial_pose.yaw: 0.0
```

适用：机器人永远从同一位置启动（充电桩、固定工位）。

方式 2: RViz 手动点 2D Pose Estimate (适合研发调试)

最灵活，每次启动手动校正；定位发散时也用它救场。缺点是依赖人眼和人手，不能自动化。

方式 3: 程序自动发布 /initialpose (适合产品部署，本项目采用)

产品上初始位姿从可靠渠道自动获取：视觉标签（ArUco / 二维码）、已知的停靠点/充电站坐标、RFID 触发的已知位置等，由程序发布到 `/initialpose`。

本项目就是这条路：AMCL 参数里 `set_initial_pose: false`，启动约 8 秒后由 `lifecycle_activator` 在激活完九个 Nav2 生命周期节点之后，向 `/initialpose` 发布出生点位姿。两个工程细节值得留意（都在 `src/mecamind_tools/mecamind_tools/lifecycle_activator.py`）：

1. **QoS 用 TRANSIENT_LOCAL**：即便 AMCL 完成订阅比发布晚，也能补收到这条初始位姿；默认 `VOLATILE` 则「错过就没了」。
2. **重复发布多次**：单发一条在启动竞态下可能丢，重复几次兜底。

三种方式不冲突：本项目平时走方式 3，定位丢了随时用方式 2 人工救。

6. 怎么查「机器人现在在地图哪」？四种方法

方法 A: 查 TF (最准确，是所有下游的最终真相)

```
ros2 run tf2_ros tf2_echo map base_footprint --ros-args -p use_sim_time:=true
```

输出：

```
At time 123.45, frame base_footprint:
- Translation: [1.234, 0.567, 0.000]
- Rotation: in Quaternion [0.000, 0.000, 0.123, 0.992]
```

(1.234, 0.567) 就是机器人当前在 map 系的位置。注意这是 `map → odom` 与 `odom → base_footprint` 两段 TF 相乘的合成结果——AMCL 只发布前一段，后一段来自里程计。

顺手可以看「账本」本身：

```
ros2 run tf2_ros tf2_echo map odom --ros-args -p use_sim_time:=true
```

运动后它呈阶梯状跳变是正常的（原理与实测见《里程计漂移与矫正》第 10.5、11 节）。

方法 B：订阅 /amcl_pose（最适合写程序）

```
ros2 topic echo /amcl_pose --once
```

AMCL 直接发布它的位姿估计，**附带协方差矩阵**——数值越小越自信。本项目 CP3 验收脚本（`nav_acceptance.py`）就是订阅它来判断「定位是否收敛」的：协方差下标 0 和 7 分别是 x、y 方差。

方法 C：RViz 里看

Fixed Frame 选 `map`，添加 TF 显示，`base_footprint` 坐标轴所在处就是机器人当前位置。再勾选 Particle Cloud：粒子抱团越紧 = 定位越自信。

方法 D: 写代码订阅 (Python)

```
import math

import rclpy
from geometry_msgs.msg import PoseWithCovarianceStamped
from rclpy.node import Node

class PoseListener(Node):
    def __init__(self):
        super().__init__("pose_listener")
        self.create_subscription(
            PoseWithCovarianceStamped, "/amcl_pose", self.pose_callback, 10
        )

    def pose_callback(self, msg):
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        # 平面运动时四元数只有 z/w 分量非零, yaw = 2*atan2(z, w)
        qz = msg.pose.pose.orientation.z
        qw = msg.pose.pose.orientation.w
        yaw = 2.0 * math.atan2(qz, qw)
        cov_x = msg.pose.covariance[0]
        cov_y = msg.pose.covariance[7]
        self.get_logger().info(
            f"位置: ({x:.2f}, {y:.2f}), 朝向: {math.degrees(yaw):.1f}°, "
            f"方差: x={cov_x:.3f} y={cov_y:.3f}"
        )

def main(args=None):
    rclpy.init(args=args)
    node = PoseListener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

两点比「网上常见版本」多想一步:

- `yaw = 2*atan2(z, w)` 只对**平面运动** (`roll = pitch = 0`) 成立, 机器人上下坡时要用完整的四元数转欧拉角;
- `/amcl_pose` 只在 AMCL 做滤波更新时发布 (车动了 `update_min_d: 0.15 m` 或转 `update_min_a: 0.15 rad` 才更新一次), **频率很低且不均匀**。要高频连续位姿, 用方法 A 的 TF (`tf2_ros.Buffer.lookup_transform`), 别守着这个话题等。

7. 常见误区速查

× 误区	☑ 正确理解
RViz 点 2D Pose Estimate 是在设置 map 原点	只是告诉 AMCL 「机器人目前在这里」，map 系分毫未动
YAML 里的 origin 是 map 坐标系的原点	origin 是地图图片左下角在 map 系中的位置；map 原点永远是 (0,0)
AMCL 会自动在整张地图上搜索机器人	默认不会 (recovery_alpha_* = 0.0)；它是局部收敛算法，必须给初值
机器人启动位置必须正好在 map 原点	可以在任何地方，但要通过 initial_pose / 2D Pose Estimate 告诉 AMCL
机器人在 map 系的位姿直接由 AMCL 提供	AMCL 只发布 map → odom；odom → base_footprint 来自里程计，最终位姿是两段相乘
initial_pose 写在哪儿都一样	Nav2 的 AMCL 支持参数写死（方式 1），但本项目走 set_initial_pose: false + 程序发布 /initialpose（方式 3），别在两处同时配置造成困惑
/amcl_pose 可以当高频定位源用	它随滤波更新低频发布；高频连续位姿请查 TF

8. 总结

一句话记住核心：

map 坐标系是建图时就固定的「死」东西；initial_pose 和 2D Pose Estimate 只是让 AMCL 在这个固定坐标系里，更快找到机器人在哪而已。

配置项速查表：

配置项 / 接口	含义	所属坐标系	在本项目的位置
地图 YAML <code>origin</code>	图片左下角像素对应的 map 坐标	map	<code>src/mecamind_tools/maps/mecamind_three_room_nav.yaml</code>
<code>initial_pose</code>	AMCL 启动时机器人的估计位置	map	<code>mecamind_named_goals.yaml</code> (由 <code>lifecycle_activator</code> 发布)
<code>/initialpose</code> 话题	运行时更新 AMCL 的估计起点	map	RViz 2D Pose Estimate / <code>lifecycle_activator</code> 都发它
<code>/amcl_pose</code> 话题	AMCL 输出的实时位姿估计 (带协方差)	map	CP3 验收脚本消费它
TF <code>map → odom</code>	漂移矫正账本 (差值)	—	AMCL 发布, 阶梯状跳变
TF <code>map → base_footprint</code>	机器人在地图上的当前位置	map	两段 TF 相乘的合成结果

读完本篇再回看两篇讲义，衔接关系是：本篇讲清「坐标系与初值」这些静态概念 → 《里程计漂移与矫正》讲 `map → odom` 运行时怎么被结算 → 主讲义把它们放进导航主线四步（加载地图 → 初始定位 → 行走矫正 → 规划跟线）。